

BoxAdmin — Fase 1: Núcleo operativo

Especificación técnica para agente de desarrollo (Claude Code)

Septiembre 2026

Contexto

Continúa sobre la base de la **Fase 0** (auth, multi-tenancy, infraestructura). Esta fase introduce las primeras entidades reales del dominio del gimnasio: **Salas, Usuarios de negocio (alumno/profesor), Packs y Turnos**, más la capacidad de asignar reservas manualmente desde un calendario admin.

Objetivo de esta fase: un admin puede configurar su salón desde cero (salas → packs → alumnos/profesores → turnos) y asignar reservas a mano, replicando el flujo operativo base de TurnoFit pero con los dos formularios de alta separados (alumno vs. profesor) y el catálogo de precios visible como sección propia, no escondida.

Criterio de éxito: un admin puede dar de alta una sala, un pack, un alumno con ese pack, un horario/turno, y asignarle una reserva manual desde el calendario — todo vía API, con las reglas de negocio base (cupos, salas con acceso) validadas en el backend.

1. Dependencias adicionales

No hay librerías nuevas relevantes en esta fase — se construye sobre NestJS + Prisma ya instalados. Sí agregar:

```
pnpm add nanoid # para generar códigos cortos (ej. slugs de sala, códigos de reserva)
```

2. Modelo de datos de esta fase (Prisma)

Agregar al `schema.prisma` existente:

```
model Sala {
  id          String    @id @default(cuid())
  tenantId    String
  tenant      Tenant    @relation(fields: [tenantId], references: [id])

  nombre      String
  activa      Boolean    @default(true)
  visibleAlumnos Boolean @default(true)
  soloCuposLiberados Boolean @default(false)
  exclusiva   Boolean    @default(false)

  cupoBase    Int?      // null = hereda de configuración general del tenant
  minMinutosCancelar Int? // null = hereda
  minMinutosAnotarse Int? // null = hereda
  listaEsperaHabilitada Boolean? // null = hereda
```

```

    createdAt DateTime @default(now())
    updatedAt DateTime @updatedAt

    usuarios      UsuarioSala[]
    turnos         Turno[]
    packs          Pack[]

    @@index([tenantId])
    @@map("salas")
}

model Perfil {
  // Datos específicos del alumno o profesor, separados de la tabla de auth (Usuario de
  ↪ Fase 0)
  id          String    @id @default(cuid())
  tenantId     String
  usuarioId    String    @unique
  usuario      Usuario   @relation(fields: [usuarioId], references: [id], onDelete:
  ↪ Cascade)

  telefono      String?
  fichaMedica   String? // dato sensible: acceso restringido por rol en el service, no
  ↪ expuesto por defecto

  // Solo aplica si el usuario es ALUMNO
  packId        String?
  pack          Pack?    @relation(fields: [packId], references: [id])
  clasesExtra   Int       @default(0)
  cancelacionesUsadas Int   @default(0)
  pagoAlDia     Boolean   @default(false)
  vigenciaDesde DateTime?
  vigenciaHasta DateTime?

  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt

  salas UsuarioSala[]

  @@index([tenantId])
  @@map("perfiles")
}

model UsuarioSala {
  // Salas con acceso de un usuario (many-to-many)
  perfilId String
  perfil  Perfil @relation(fields: [perfilId], references: [id], onDelete: Cascade)
  salaId  String
  sala    Sala   @relation(fields: [salaId], references: [id], onDelete: Cascade)

  @@id([perfilId, salaId])
  @@map("usuarios_salas")
}

enum TipoPack {
  MENSUAL
  TOTAL
}

model Pack {
  id          String    @id @default(cuid())

```

```

tenantId      String
tenant        Tenant    @relation(fields: [tenantId], references: [id])

nombre        String    // ej. "8 clases"
salaId        String?   // null = todas las salas
sala          Sala?     @relation(fields: [salaId], references: [id])
tipo          TipoPack  @default(MENSUAL)
precio        Decimal?  @db.Decimal(10, 2) // null = "a consultar"
clasesPorMes  Int?      // aplica si tipo = MENSUAL
clasesTotales Int?      // aplica si tipo = TOTAL
cancelacionesPermitidas Int? // null = el alumno mantiene su propio contador manual
activo        Boolean   @default(true)

createdAt DateTime @default(now())
updatedAt DateTime @updatedAt

perfiles Perfil[]

@@index([tenantId])
@@map("packs")
}

enum OrigenReserva {
    ADMIN
    ALUMNO
    RUTINA
    PRUEBA
    LISTA_ESPERA
    EXTRA
}

model Turno {
    id          String    @id @default(cuid())
    tenantId    String
    tenant      Tenant    @relation(fields: [tenantId], references: [id])
    salaId      String
    sala        Sala      @relation(fields: [salaId], references: [id])

    nombre      String    // ej. "Pilates", "Circuito" -- en Fase 4 se agrega profesorId
    fecha       DateTime  @db.Date
    horaInicio  String    // "18:00" -- string simple; se puede migrar a DateTime completo si
    ↪ hace falta ordenar por rango horario
    horaFin     String
    cupo        Int

    createdAt DateTime @default(now())
    updatedAt DateTime @updatedAt

    reservas Reserva[]

    @@index([tenantId, salaId, fecha])
    @@map("turnos")
}

model Reserva {
    id          String    @id @default(cuid())
    tenantId    String
    tenant      Tenant    @relation(fields: [tenantId], references: [id])
    turnoId     String
    turno       Turno      @relation(fields: [turnoId], references: [id])

```

```

perfilId    String
perfil      Perfil      @relation(fields: [perfilId], references: [id])

origen      OrigenReserva
esPrueba    Boolean     @default(false)
pagoRealizado Boolean    @default(false)

canceladaEn    DateTime?
cancelacionTipo String? // "recuperable" | "definitiva"

createdAt DateTime @default(now())
updatedAt DateTime @updatedAt

@@index([tenantId, turnoId])
@@index([tenantId, perfilId])
@@map("reservas")
}

```

Ojo con el rol Perfil vs Usuario: Usuario (Fase 0) es solo identidad + auth + rol. Perfil guarda todo lo específico de negocio (pack, teléfono, ficha médica, clases extra). Esta separación es la que resuelve directamente el problema que viste en TurnoFit: el alta de un profesor ya no arrastra campos de alumno sin sentido, porque Perfil es opcional/parcial y los DTOs de alta se validan distinto según el rol del Usuario asociado.

Migración:

```
pnpm dlx prisma migrate dev --name fase1_nucleo_operativo
```

3. Estructura de módulos NestJS

```

src/
├── salas/
│   ├── salas.module.ts
│   ├── salas.controller.ts
│   ├── salas.service.ts
│   └── dto/
├── usuarios/
│   ├── usuarios.module.ts
│   ├── usuarios.controller.ts
│   ├── usuarios.service.ts
│   └── dto/
│       ├── crear-alumno.dto.ts
│       └── crear-profesor.dto.ts
├── packs/
│   ├── packs.module.ts
│   ├── packs.controller.ts
│   ├── packs.service.ts
│   └── dto/
├── turnos/
│   ├── turnos.module.ts
│   ├── turnos.controller.ts
│   ├── turnos.service.ts
│   └── dto/
└── reservas/
    ├── reservas.module.ts

```

```

├─ reservas.controller.ts
├─ reservas.service.ts
└─ dto/

```

4. Endpoints de esta fase

4.1 Salas

Método	Ruta	Rol requerido
POST	/salas	ADMIN_SALON
GET	/salas	cualquier rol autenticado del tenant
GET	/salas/:id	cualquier rol autenticado
PATCH	/salas/:id	ADMIN_SALON
DELETE	/salas/:id	ADMIN_SALON (baja lógica: activa=false, nunca DELETE físico si tiene turnos asociados)

4.2 Usuarios (dos DTOs de entrada, un mismo recurso)

Método	Ruta	Notas
POST	/usuarios/alumnos	DTO propio: nombre, email, packId (opcional), salas con acceso, pago inicial. No pide “clases mensuales” si el pack ya lo define.
POST	/usuarios/profesores	DTO propio: nombre, email, salas con acceso. Sin campos de pack/clases/cancelaciones — resuelve directamente la limitación que viste en TurnoFit.
GET	/usuarios?tipo=alumno\ profesor	Lista de usuarios, igual que “Administrar Usuarios”
GET	/usuarios/:id	Detalle completo (perfil + salas + pack)
PATCH	/usuarios/:id	Edición de perfil
PATCH	/usuarios/:id/salas	Actualiza salas con acceso — debe rechazar guardar con 0 salas y devolver un error claro , o al menos loguear una advertencia en HistorialAccion, para no repetir el bug silencioso de “Sin sala” que encontraste en Wellness
POST	/usuarios/:id/reset-password	Resetea contraseña (equivalente a “Clave”)
DELETE	/usuarios/:id	Baja lógica

4.3 Packs (catálogo, visible como sección propia — no escondida)

Método	Ruta
POST	/packs
GET	/packs?salaId=
PATCH	/packs/:id
DELETE	/packs/:id (baja lógica si tiene perfiles asociados)

4.4 Turnos

Método	Ruta
POST	/turnos
GET	/turnos?desde=&hasta=&salaId=&soloLibres=
PATCH	/turnos/:id
DELETE	/turnos/:id

4.5 Reservas (asignación manual, equivalente al botón “+” del calendario)

Método	Ruta
POST	/turnos/:turnoId/reservas
DELETE	/reservas/:id?tipo=recuperable\ definitiva
PATCH	/reservas/:id/reasignar

5. Reglas de negocio a validar en el service layer (no solo en la DB)

- Al crear una reserva, contar reservas activas del turno y comparar contra `turno.cupo` antes de insertar (usar una transacción con `SELECT ... FOR UPDATE` o el nivel de aislamiento `Serializable` de Prisma para evitar condiciones de carrera si dos requests llegan al mismo tiempo — esto importa de verdad en horarios populares).
- Al dar de alta un alumno, si no se asigna ninguna sala, devolver un warning explícito en la respuesta (no solo guardar silenciosamente como hacía `TurnoFit`).
- `Perfil.cancelacionesUsadas` se incrementa solo cuando la cancelación la origina el alumno dentro de la ventana de reglas — las cancelaciones hechas por el admin no cuentan contra ese contador (así lo indica el documento de relevamiento).
- Cada alta, edición de salas, cancelación y reasignación debe escribir un registro en `HistorialAccion` (tabla creada en Fase 0) con entidad, entidadId, accion y el `usuarioId` que ejecutó la acción.

6. Checklist de aceptación de la Fase 1

- ☐ Se puede crear una sala y configurarla con sus reglas propias (o heredar del tenant).
- ☐ Se puede crear un pack en el catálogo de Precios, independiente de cualquier alumno puntual.
- ☐ El alta de alumno y de profesor usan DTOs y validaciones distintas — un profesor nunca debe poder recibir un error de “clases mensuales requerido”.
- ☐ Un alumno sin salas asignadas genera un warning visible en la respuesta del API.
- ☐ Se puede crear un turno puntual y asignarle una reserva manual respetando el cupo.
- ☐ Intentar reservar un turno lleno devuelve 409, no un registro fuera de cupo.
- ☐ Cancelar una reserva como “recuperable” devuelve la clase al perfil; como “definitiva”, no.

- ☐ Reasignar una reserva mueve correctamente el registro y valida cupo del turno destino.
 - ☐ Cada operación relevante queda registrada en `historial_acciones`.
 - ☐ Tests e2e cubren: alta de sala, alta de pack, alta de alumno con pack, alta de profesor, creación de turno, reserva manual, cancelación recuperable y definitiva.
-

7. Qué NO entra en esta fase

Rutinas fijas, armado automático del mes, detección de conflictos, self-service del alumno (PWA), profesor como entidad vinculada al turno, pagos con comprobante, y estadísticas — todo eso llega en fases posteriores. Acá el admin todavía hace **todo a mano**, y está bien que así sea: es la base sobre la que se automatiza después.